

Sanskrit-to-English Neural Machine Translation

Natural Language Understanding - Assignment 2

Roll No: G25AIT1114

Code (public): https://github.com/ghewaripp/G25AIT1114_NLU_2

1. Introduction

This report describes a custom sequence-to-sequence (seq2seq) system for translating Sanskrit into English. Sanskrit is a low-resource, morphologically rich language with free word order and heavy compounding (samasa) and sandhi, which makes translation difficult. Working from a parallel corpus of 10,000 training pairs, we build an attention-based recurrent encoder-decoder trained end-to-end from scratch and decode with beam search. The model is evaluated with corpus BLEU and BERTScore (F1), and we additionally report efficiency (parameter count and test-set inference time). No pre-trained translation model is used; the only pre-trained component is RoBERTa-large, used solely inside the BERTScore metric.

2. Architecture

Encoder. Sanskrit input is tokenised at the character level and embedded (128-d). A single-layer bidirectional GRU (hidden size 256 per direction) reads the sequence; inputs are packed with `pack_padded_sequence` so padding does not affect the recurrence. The final forward and backward states are concatenated (512-d) and projected through a tanh linear layer to initialise the decoder state (256-d).

Attention. We use additive (Bahdanau) attention. At each step the previous decoder hidden state is scored against every encoder output via a feed-forward layer (tanh then a linear projection to a scalar). Scores at padded source positions are masked before a softmax, giving a distribution over source characters. The context vector is the attention-weighted sum of encoder outputs.

Decoder. A single-layer GRU consumes the concatenation of the previous target-word embedding (128-d) and the attention context (512-d). The output layer takes the concatenation of the GRU output, the context vector, and the input embedding, and projects to the English vocabulary. English is tokenised at the word level.

Beam search. At inference we decode with beam search (width 5) using length-normalised scoring (penalty exponent $a=0.7$) to avoid a bias toward very short outputs. During search we forbid the pad, unk and sos tokens, and forbid the eos token on the first step so every hypothesis contains at least one real word.

3. Training Procedure

Preprocessing and tokenization. Each Sanskrit-English file pair is merged on the shared `Source_id` column; empty rows are dropped and whitespace stripped (UTF-8/BOM handled with `utf-8-sig`). Source text is split into characters; target text is lowercased and whitespace-tokenised. Four special tokens (pad, unk, sos, eos) occupy indices 0-3. The source vocabulary is 194 characters (min frequency 1); the target vocabulary is 7,072 words, built with min frequency 2 to drop noisy singletons. Sequences are truncated to 180 source characters / 60 target words and padded per batch.

Hyperparameters are summarised below.

Setting	Value
Source / target tokenization	character-level / word-level
Source vocab / target vocab	194 / 7,072 (target min freq = 2)
Max source / target length	180 chars / 60 words
Embedding dimension	128

Encoder	Bidirectional GRU, hidden 256 (x2)
Decoder	GRU, hidden 256
Attention	Bahdanau (additive), padding-masked
Dropout	0.3
Batch size	64
Optimizer	Adam, learning rate 3e-4
LR scheduler	ReduceLROnPlateau (factor 0.5, patience 2)
Gradient clipping	1.0 (max norm)
Teacher forcing ratio	0.5
Loss	Cross-entropy (ignore pad token)
Epochs	30 (best checkpoint by dev loss)
Decoding	Beam search, width 5, length penalty $\alpha=0.7$
Total parameters	8,884,640

Optimization and regularization. We minimise token-level cross-entropy (padding ignored) with Adam. Gradients are clipped to norm 1.0. Teacher forcing with ratio 0.5 mixes gold and predicted tokens during training to reduce exposure bias. Regularization comes from dropout (0.3), the min-frequency-2 target vocabulary, and early model selection: the checkpoint with the lowest dev loss is saved and used for all evaluation. The learning rate is halved when dev loss plateaus.

4. Results

Quantitative results on the dev and test sets:

Metric	Dev	Test
BLEU (NLTK corpus_bleu, default weights)	0.0286	0.0271
BERTScore F1 (rescale_with_baseline=True)	0.0478	0.0519
Total parameters	8,884,640	8,884,640
Inference time (test set)	-	232.34 s / 1000 (232.3 ms/sent)

Training curves. Training loss decreases monotonically from 6.49 (epoch 1) to 4.22 (epoch 30), while dev loss falls quickly from 5.78 to about 5.25 by epoch 9 and then plateaus, reaching its minimum (about 5.25, perplexity about 191) around epoch 18. The widening gap between train and dev loss indicates the model begins to overfit after roughly 10 epochs; the best-dev checkpoint is therefore selected rather than the final epoch.

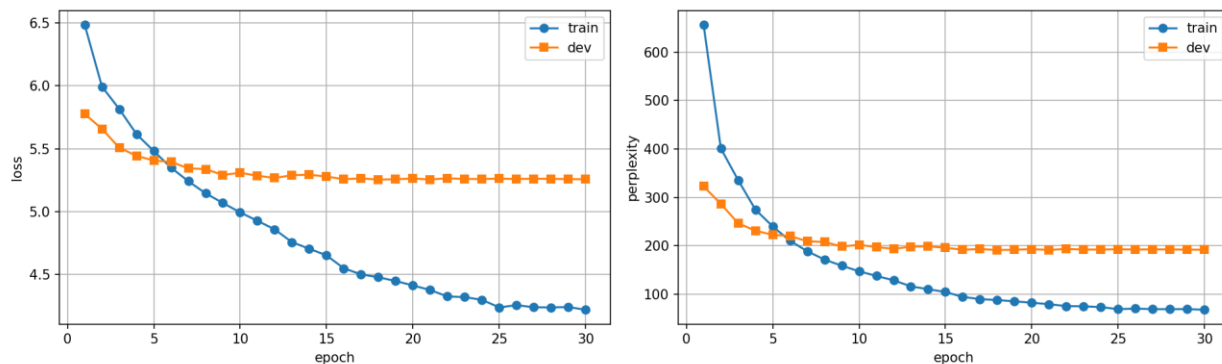


Figure 1: Training and validation loss (left) and perplexity (right) over 30 epochs.

5. Translation Examples and Error Analysis

Representative dev-set outputs (source, reference, prediction):

Example 1

SA: ते वीराः ।

REF: Those are brave men.

PRED: those two are drums.

Example 2

SA: एते तिथी ।

REF: These two are dates.

PRED: these two are

Example 3

SA: यूयं भक्तियोगं पठथ।

REF: You all are studying bhakti yoga.

PRED: you all the

Example 4

SA: पाठेऽस्मिन् वयम्, awk ... if, else, else if ...

REF: In this tutorial, we will learn about if, else, else if in awk.

PRED: in this tutorial, we will learn how to create a file and

Example 5

SA: 'इन्फ़ैनेट लूप' ...

REF: Infinite loop can cause the system to become unresponsive.

PRED: the output is the the the

Example 6

SA: ततस्तस्य गात्रे ...

REF: And they spit upon him, and took the reed, and smote him on the head.

PRED: and when he was to the the of the

Error patterns. (i) Correct syntax, wrong content (Ex. 1, 3): the model reliably reproduces English sentence frames but substitutes incorrect content words, so it learns surface syntax better than lexical alignment. (ii) Fluent but hallucinated or truncated (Ex. 2, 4): outputs are grammatical and domain-appropriate (much of the training data is software Spoken-Tutorial transcripts, so the decoder is biased toward that register) but drift from the source meaning or stop early. (iii) Repetition and degeneration on

long inputs (Ex. 5, 6): for long or rare sentences the decoder loops as attention diffuses across a long character sequence. These behaviours are consistent with a small RNN model trained on limited data and explain the low BLEU: n-gram overlap is sparse even when the output is readable.

6. Experiments

- Vocabulary threshold: using min-frequency 2 for English (versus 1) reduced the target vocabulary and the number of unpredictable singleton words, improving stability.
- Decoding strategy: the largest practical gains came from decoding, not architecture. A plain beam search produced many empty or degenerate outputs (dev BLEU about 0.022, BERTScore F1 about 0.42). Adding length-normalised scoring ($\alpha=0.7$) and forbidding the unk/pad/sos tokens and premature eos removed the empties and raised dev BLEU to 0.0286 and BERTScore F1 to 0.0478.
- Toolchain compatibility: a version incompatibility between bert-score 0.3.x and the newest transformers (v5) broke the metric; pinning transformers==4.44.2 and tokenizers==0.19.1 resolved it without affecting the model.

7. Discussion

Design choices. Character-level encoding for Sanskrit keeps the source vocabulary tiny (194 symbols), sidesteps out-of-vocabulary issues from compounding and sandhi, and lets the encoder see sub-word structure. Word-level decoding keeps English targets short and readable. Bahdanau attention is the standard alignment mechanism for RNN seq2seq, and beam search with a length penalty is a simple, effective decoder.

Challenges. (i) Low resource: 10k pairs is small for open-domain translation; (ii) mixed domains and code-switched tokens (for example awk, Ctrl, void) in the data; (iii) exposure bias and degeneration/looping at inference; (iv) evaluation-library version conflicts.

Limitations. The char-to-word design loses explicit source word semantics; a single-layer GRU has limited capacity; there is no coverage mechanism to prevent repetition; and the train/dev gap shows overfitting. Consequently absolute BLEU/BERTScore are low, though outputs are often fluent.

Future work. Sub-word (BPE/SentencePiece) tokenization on both sides, a coverage or no-repeat-n-gram constraint to curb looping, label smoothing and scheduled sampling, a deeper decoder or a Transformer encoder-decoder, and back-translation-style data augmentation.

8. References and Pre-trained Model Disclosure

Pre-trained models used. The translation model is trained entirely from scratch on the provided dataset. The only pre-trained model used anywhere is RoBERTa-large, invoked solely by the bert-score library to compute the BERTScore metric; it is not part of the translation system. NLTK is used for BLEU.

Code repository. Full code (public): https://github.com/ghewaripp/G25AIT1114_NLU_2

- Bahdanau, Cho, Bengio (2015). Neural Machine Translation by Jointly Learning to Align and Translate. ICLR.
- Sutskever, Vinyals, Le (2014). Sequence to Sequence Learning with Neural Networks. NeurIPS.
- Cho et al. (2014). Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. EMNLP.
- Papineni, Roukos, Ward, Zhu (2002). BLEU: a Method for Automatic Evaluation of Machine Translation. ACL.
- Zhang, Kishore, Wu, Weinberger, Artzi (2020). BERTScore: Evaluating Text Generation with BERT. ICLR.

- Paszke et al. (2019). PyTorch. NeurIPS. Bird, Klein, Loper (2009). Natural Language Processing with Python (NLTK).